



牛客竞赛

AC.NOWCODER.COM

2025 牛客 暑期多校训练营 5

Moon Cat, Stump

HR.NOWCODER.COM



概况

- 入门: E, I;
- 简单: H, K, J;
- 较简单: A, C, D;
- 中等: B, L, M;
- 较难: F, G;

A - Entangled Coins

题意

- 现有一个恒在范围 $[0, n]$ 内的数 x ，每步操作可使其变为 $s - wl + wr$ ，其中 $wl + wr = k$ 且 $wl \in [0, s]$, $wr \in [0, n - s]$;
- 或，每步操作可使 x 变为范围 $[|x - k|, n - |(n - x) - k|]$ 之间任意与 $x + k$ 奇偶性相同的数;
- 求使 x 从 s 变为 t 所需的最小步数。
- $1 \leq k \leq n \leq 10^9, 0 \leq s, t \leq n$ 。

A - Entangled Coins

- 当 k 为奇数时每步操作 x 的奇偶性都会改变, “使 x 变为 t ”对操作次数的奇偶性有要求, 自然地应想到将操作次数为奇/偶的情况分开讨论, 以及研究连续操作两次的效果:
- 设操作前的数为 x , 操作后的数为 y , 由对称性不妨设 $x \leq \lfloor \frac{n}{2} \rfloor$, 并另假设有 $k \leq \lfloor \frac{n}{2} \rfloor$;
- 我们可以先使第一步的 $wl_1 = 0, wr_1 = k$, 第二步的 $wl_2 = k, wr_2 = 0$, 以使 x 经两步到达 $y = x$;

A - Entangled Coins

- 在“到达 x ”的方案的基础上逐步调大 wl_1 ，有 $y = x - 2wl_1$ 和限制 $wl_1 \leq \min(x, k)$ ，则 x 经两步可到达 $[\max(x - 2k, 0), x]$ 间任意与 x 奇偶性相同的位置；
- 在“到达 x ”的方案的基础上逐步调大 wr_2 ，有 $y = x + 2wr_2$ 和限制 $wr \leq \min(n - (x + k), k)$ ，则 x 经两步可到达 $[x, \min(x + 2k, n)]$ 间任意与 x 奇偶性相同的位置；
- 即： $k \leq \lfloor \frac{n}{2} \rfloor$ 时， x 经两步可到达 $[\max(x - 2k, 0), \min(x + 2k, n)]$ 间任意与 x 奇偶性相同的位置。

A - Entangled Coins

- 如此，当 $k \leq \lfloor \frac{n}{2} \rfloor$ 时， x 能以 $2c$ 步到达 y 的充要条件为 $[(x + y) \bmod 2 \equiv 0] \wedge [x - c \cdot 2k \leq y \leq x + c \cdot 2k]$ ；
- 另当 $k > \lfloor \frac{n}{2} \rfloor$ 时，考虑原题意，“选中 k 枚硬币翻转”相当于“选中 $n - k$ 枚硬币翻转，再使所有硬币翻转”；“两次选中 k 枚硬币翻转”相当于“两次选中 $n - k$ 枚硬币翻转，再两次使所有硬币翻转（相抵消）”。再因此时 $n - k \leq \lfloor \frac{n}{2} \rfloor$ ，将上一段结论中的 k 替换为 $n - k$ 即可。
- 整理：取 $K = \min(k, n - k)$ ， x 能以 $2c$ 步到达 y 的充要条件为 $[(x + y) \bmod 2 \equiv 0] \wedge [x - c \cdot 2K \leq y \leq x + c \cdot 2K]$ 。

A - Entangled Coins

- 我们继续“将操作次数为奇/偶的情况分开讨论”：
 - 操作次数为偶 ($2c$) 时，先判断奇偶性，再取最小的 c 使得 $s - c \cdot 2K \leq t \leq s + c \cdot 2K$ 即可，即取 $c = \lceil \frac{|s-t|}{2K} \rceil$ ；
 - 操作次数为奇 ($2c+1$) 时，先判断奇偶性，再取出 s 经一步操作的可达区间 $[l, r]$ ，再取最小的 c 使得 $l - c \cdot 2K \leq t \leq r + c \cdot 2K$ 即可，即取 $c = \max(\lceil \frac{l-t}{2K} \rceil, \lceil \frac{t-r}{2K} \rceil, 0)$ ；
- 最后在两情况的答案（若无解取正无穷）中取较小值输出，均为正无穷则输出 -1 ；
- 单测复杂度 $O(1)$ 。

B - Extra Training

题意

- 给定一长度为 n ，字符集大小为 $V = 52$ 的字符串 S 和一参数 k ，求 k 个可空 S 子串拼接可产生的本质不同可空串数，答案对 998244353 取模。
- $1 \leq n \leq 5 \times 10^5, 1 \leq k \leq 10^9$ 。

B - Extra Training

- 我们希望每个本质不同的串只被拼接组合一次，那不妨只取最快的那种拼接方式；
- 若要拼接串 R ，我们需要不断取 R 最长的，在 S 子串中出现的前缀，并将该前缀取出去除；这意味着若定义拼接串 R 的第 i 个串为 s_i ，则存在一数 c ($0 \leq c \leq k$) 使得 $s_{i>c}$ 全为空串， $s_{i \leq c}$ 全为非空串，且对于任意 $1 \leq i < c$ ， s_i 为 S 的子串，而 s_i 末尾加上 s_{i+1} 的首字符即不再是 S 的子串——拼接的子串仅对下一个拼接子串的首字符有要求。

B - Extra Training

- 不妨倒着选串，仅维护已拼接 $0 \leq i \leq k$ 个串，拼接子串后缀的首字符编号为 $w = [0, V]$ 分别的方案数 $dp_{i,w}$ （记初始首字符编号为 V ，可前接任何子串）；
- 为了递推 dp 值，需要预先计算各“允许拼接后缀原首字符为 i ，且以字符 j 为新首字符”，即“首字符为 i ，末尾加 j 后不为 S 子串”的 S 子串个数 $tr_{i,j}$ 形成递推矩阵：
 - 可用 S 的 SAM (后缀自动机) 利用 SAM 总边数不超过 $3n$ 的性质改进求和方法，以时空 $O(nV)$ 的复杂度得到 tr ；若对 SAM 的每个节点以 $O(V^2)$ 直接对每个 $tr_{i,j}$ 贡献， $O(nV^2)$ 的时间复杂度是不可取的。

B - Extra Training

■ 最后计算答案:

■ 可以求 $\sum_{i=0}^V Tr_{V,i}$, 其中 $Tr = \sum_{i=0}^{k-1} tr^i$;

■ $Tr_{V,V}$ 有意义, 代表空串;

■ 记 $A_j = \sum_{i=0}^{j-1} tr^i$, 并先得到 A_1 和 tr^1 ;

■ A_j 和 tr^j 能以 $O(V^3)$ 递推得到 A_{j+1} 和 tr^{j+1} ; 又能以 $O(V^3)$ 倍增得到 A_{2j} 和 tr^{2j} ; 由此对于任意 $k > 0$, $Tr = A_k$ 和 tr^k 都能由 A_1 和 tr^1 以 $O(V^3 \log k)$ 得到。

B - Extra Training

- 也可以将 tr 的两维再扩大 1 到 $V+1$, 编号为 $V+1$ 代表新拼接串首字符为空;
 - 问题转化为: $k+1$ 个 S 可空子串拼接, 且最后一个串必为空串时, 可得到的本质不同串数;
 - 对所有 $0 \leq i \leq V+1$ 取 $tr_{i,V+1} = 1$, 表示拼接空串; 再矩阵快速幂 $O(V^3 \log k)$ 求 $Tr' = tr^{k+1}$, $Tr'_{V,V+1}$ 即为答案。
- 总复杂度 $O(nV + V^3 \log V)$ 或 $O(nV)$ 。

C - Array Deletion Game

题意

- 给定一长度为 N 的正整数序列 A ，并询问 Q 次不同的 s ，定义状态二元组 (l, r) 不合法当且仅当 $[l > r] \vee [\sum_{i=l}^r A_i \leq s]$ ；
- 状态 (l, r) 可选择转移到 $(l+1, r)$ 或 $(l, r-1)$ ；两人从状态 $(1, n)$ 起（保证其合法）轮流转移，不能转移到合法状态者败，问对于每个 s 先后手谁胜。
- $1 \leq N \leq 10^5, 1 \leq A_i \leq 10000, 1 \leq Q \leq 10^5, 1 \leq s < \sum A$ 。

C - Array Deletion Game

- 定义无法从其开始先手并获胜的状态为“败态”，否则为“胜态”；
- 每当我们确定一状态 (l, r) 为败态时，即可确定 $(l-1, r)$ 和 $(l, r+1)$ 为可转移至败态的胜态，以及 $(l-1, r+1)$ 为无法转移至其他败态的败态，并循环推导下去，确定 3 串 $l+r$ 为 3 个连续定值的状态的胜败态；
- 由此，在推导过程中的任意时刻， $r-l$ 相同的各合法状态的胜败态要么全部未知，要么由其中 $r-l$ 最小的合法状态一次性确定；求合法状态 (l, r) 的胜败态即求 $(l' = l+d, r' = r-d)$ 的胜败态， d 满足 $d \geq 0$ ，状态 $(l+d, r-d)$ 合法，以及状态 $(l+d+1, r-d-1)$ 不合法，可二分得到。

C - Array Deletion Game

- 而对于状态 (l', r') :
 - 二分找到 $fl \geq l'$, 使得 (fl, r') 合法而 $(fl + 1, r')$ 不合法; 当 $fl > l'$ 时, 由 $(l' + 1, r' - 1)$ 不合法而 $(l' \geq l' + 1, r' - 1)$ 不合法, (fl, r') 为必败态, $(fl - 1, r')$ 为必胜态.....结合 $fl = l'$ 的情况, 当 $fl - l' \bmod 2 \equiv 1$ 时, (l', r') 为必胜态并向 $(l' + 1, r')$ 转移可取胜, 否则不应向 $(l' + 1, r')$ 转移;
 - 类似地, 二分找到 $fr \leq r'$, 使得 (l', fr) 合法而 $(l', fr - 1)$ 不合法; 当 $r' - fr \bmod 2 \equiv 1$ 时, (l', r') 为必胜态并向 $(l', r' - 1)$ 转移可取胜, 否则不应向 $(l', r' - 1)$ 转移;
 - 状态 (l', r') 为必胜态当且仅当 $[fl - l' \bmod 2 \equiv 1] \vee [r' - fr \bmod 2 \equiv 1]$ 。



C - Array Deletion Game

预处理 A 的前缀和以快速计算 $\sum_{i=l}^r A_i$, 以 3 次二分回答每个询问, 总复杂度为 $O(N + Q \log N)$ 。

D - Prime XOR Permutation

题意

- 给定一正整数 N ，构造一含 0 到 $N-1$ 共 N 个整数各一个的序列，使得其任意邻项间的异或和为质数。
- $1 \leq N \leq 10^6$ ，数据 $\sum N \leq 1e6$ ，组数 $1 \leq T \leq 2 \times 10^5$ 。

D - Prime XOR Permutation

0. 手模几个小数字，会发现除了 $n = 2$ 确实不可能，其他都能模出来。
1. 首先考虑 $n = 2^k$ 的形式：
- $n = 4$ 时，有 0 3 1 2 这组解。
 - 那么 $n = 8$ 时，我们可以发现 $[4, 7]$ 和 $[0, 3]$ ，这两个区间的差别只有前者的每个数比后者的每个数多个最高位的 1，那么我们仿照 $n = 4$ 的结构可以拼出 4 7 5 6，但显然 0 3 1 2 不能和 4 7 5 6 直接组合。
 - 考虑如何过度，我们把 $n = 4$ 时，最后一位换成 3，就可以拼接了，那怎么换呢？
 - 有性质 $a \oplus b = (a \oplus c) \oplus (b \oplus c)$ ，所以我们只需要把 $n = 4$ 的解整体 $\oplus (2 \oplus 3)$ ，即可保证最后一位为 3，且前面也合法。

D - Prime XOR Permutation

- 此时我们就有了办法从 $n = 2^k$ 扩展到 $n = 2^{k+1}$ ，即先把前面的抄过来然后首位多加个 1 放到后面去，然后调整中间的异或和为质数，再让前面整体异或至符合答案。
- 那么能否保证中间的异或和一定能调整为质数呢？这显然是可以的，因为前一半的最后一位可以在 $[0, 2^k - 1]$ 任选，且后一半的首位 $< 2^{k+1}$ ，也就是说中间的异或和调整范围为 $[2^k, 2^{k+1} - 1]$ ，显然这个区间存在质数。

D - Prime XOR Permutation

2. 考虑 $n \equiv 0 \pmod{4}$ 的形式

- 其构造方式和 $n = 2^k$ 扩展到 $n = 2^{k+1}$ 其实极为相似。
- 例如 $n = 2^k + 2^d$ ($k > d > 1$) 的构造方案就是，把原先 $n = 2^d$ 的答案，整体 $+ 2^k$ ，放到后面去，中间的异或和为质数，再让前面整体异或至符合答案。

3. 考虑 $n \equiv 1 \pmod{4}$ 以及 $n \equiv 3 \pmod{4}$ 的形式

- 把他们补成 2 的形式，然后把最后多出来的删掉即可。

D - Prime XOR Permutation

4. 考虑 $n \equiv 2 \pmod{4}$ 的形式

- 这里讲一种不需要在前面的基础上新增加质数性质的方案。
- 我们回到 $n = 2^k$ 扩展到 $n = 2^{k+1}$ 的方案，既然答案可以从单边拼接得到，那同样也可以通过双边拼接得到。
- 既然要双边拼接，那可以考虑一下对称性。
- 我先在这里放一根对称线“|”，然后我让对称线的两边除了最低位，都是相同的。
- 例如 $n = 4$ 时，有 $0\ 2\ |\ 3\ 1$ 。(虽然最中间的异或和不是质数，但我们先不管)
- $n = 8$ 时，有 $7\ 5\ 0\ 2\ |\ 3\ 1\ 4\ 6$ 。

D - Prime XOR Permutation

- 那么，我们就可以发现对于对称组合 $(0, 1), (2, 3), (4, 5), (6, 7), \dots, (2^k, 2^k + 1), \dots$ 。每个组合分放在两边，可以使两边对称位置的异或和都是一样的。
- 也就是说我们用之前的逻辑处理半边，另一边对称就行了。
- 这样就解决了朴素做法中 $n \equiv 2 \pmod{4}$ 最后两个数比较难处理的问题。
- 至于最中间的问题，在处理完其他后，

... 7 5 0 2 | 3 1 **4 6** ...

改成

... 7 5 0 2 | 1 3 **4 6** ...

即可。

E - Mysterious XOR Operation

题意

- 对两数 x 和 y 进行 $\oplus_m$ “神秘异或操作”的结果为使 $x \oplus y$ “二进制由低到高位排在第偶数个的 1 均变为 0”后的结果；
- 给出一长 N 的非负整数序列 A ，计算 $\sum_i^N \sum_{j>i}^N A_i \oplus_m A_j$ ；
- $2 \leq N \leq 10^5, 0 \leq A_i \leq 10^8$ 。

E - Mysterious XOR Operation

- 我们可以按位处理，计算每一位对最终结果的贡献。关键在于确定在什么情况下第 w 位的异或结果会被保留。
- 对于第 w 位，当 $x \oplus y$ 在该位为 1 时，需要满足 $x \oplus y$ 的低 $w - 1$ 位中有偶数个 1，该位的 1 才会被保留。
- 设 x 的低 $w - 1$ 位有 cnt_x 个 1， y 的低 $w - 1$ 位有 cnt_y 个 1，则当且仅当 cnt_x 和 cnt_y 的奇偶性相同时， $x \oplus y$ 的低 $w - 1$ 位有偶数个 1。
- 证明如下：设低 $w - 1$ 位中， x 和 y 同时为 1 的位有 k 个，则 $x \oplus y$ 的低 $w - 1$ 位中 1 的个数为 $(cnt_x - k) + (cnt_y - k) = cnt_x + cnt_y - 2k$ 。显然，当 cnt_x 和 cnt_y 同奇偶时，该值为偶数。

E - Mysterious XOR Operation

- 因此，对于第 w 位，我们统计所有数对 (i, j) 满足：
 1. x_i 和 x_j 在第 w 位不同
 2. x_i 和 x_j 的低 $w - 1$ 位的 1 的个数的奇偶性相同
- 具体实现时，可以预处理每个数的低 $w - 1$ 位的 1 的个数的奇偶性，然后按位统计满足条件的数对数目，最后计算该位的贡献。
- 时间复杂度 $O(n \log \max(A_i))$ 。

F - Grid Survival

题意

- $n \times m$ 黑白网格棋盘上有一棋子，以由您决定向上/下 (列向) 移动一格，由对手决定向左/右 (行向) 移动一格的顺序循环移动；对棋子从黑/白网格开始的两类情况，分别以最小总代价激活 k 个特殊格中的若干个，使得任意指定颜色位置的棋子都能在有限步内落入激活格中（可能无解）。
- $2 \leq n, m \leq 10^6, 0 \leq k \leq \min(2 \times 10^5, n \times m)$ ，数据组数 $\leq 2 \times 10^5$ 。

F - Grid Survival

- 为方便表述，约定行从上到下，列从左到右。
- 先考虑：在确定棋子起始颜色和选中的特殊格集合后，如何判断棋子在所有位置都无法避开激活格：
 - 对手想让棋子不在任何时刻走到位置 (l, c) ，即不能主动将棋子走到位置 (l, c) 或 $(l-1, c), (l+1, c)$ ；现在只需要关心对手能走到的格子，即选定颜色的格子。
 - 称对手将棋子放置/主动走到之，您即有策略（或直接）获胜的位置为“必胜格”（称每个激活格扩为的以上三格为“确定必胜格”）；对应地，称对手将棋子放置/主动走到之，对手即有策略永远避开激活格的位置为“必败格”。

F - Grid Survival

- 当存在一条长 m 的“必败格路径” a 从上而下以斜折线贯穿棋盘（设第 i 个格子位置为 (l_i, c_i) ，有 $[\forall 1 \leq i \leq n \rightarrow l_i = i] \wedge [\forall 1 \leq i < n \rightarrow |c_{i+1} - c_i| = 1]$ ）时，对手可以永远将棋子位置控制在该路径上而获胜。
- 当 $(l, c-1), (l, c+1)$ 均为必胜格或不存在时， $(l-1, c), (l+1, c)$ 若存在也为必胜格（策略显然）；以该策略尽可能地寻找必胜格后，记每行 i 从左端起连续的最靠右必胜格位置为 L_i ，若某 $L_i \geq m-2$ ，则该行全为必胜格，进而所有格均为必胜格；否则必有“性质 A ”($\forall 1 \leq i < n \rightarrow |L_{i+1} - L_i| = 1$)，且各 $(i, L_i + 2)$ 形成的“轮廓线”为一条必败格路径。

F - Grid Survival

- 为了获胜，需要截断每条必败格路径，不断扩张轮廓线，直到某行全部为必胜格为止。
- 考虑维护 L 序列：
 - 初始所有 L 的值为 0 或 -1 ;
 - 不断使用在轮廓线上的“某激活格扩出的确定必胜格 (l', c') ” (在轮廓线上即满足 $c' = L_l + 2$); 若 $c' + 2 \geq m$ 则确定胜利; 否则使 $L_l = c'$ ，并由“性质 A”更新其他 L 和轮廓线，并寻找“轮廓线新的部分” (可被两条射线 $L_1: l + c = l' + c' \wedge c \leq c' + 2$ 和 $L_2: l - c = l' - c' \wedge c \leq c' + 2$ 的交集覆盖) 所包含的确定必胜格，继续循环;

F - Grid Survival

- 以上策略可以简化为：寻找一个确定必胜格序列，满足：
 - 记该序列为 b ，长度为 len ，第 i 个格子 b_i 的位置为 (l_i, c_i) ；
 - $c_1 \leq 2, c_{len} \geq m - 1$ ；
 - $\forall 1 \leq i < len \rightarrow [c_{i+1} \leq c_i + 2] \wedge ([l + c = l' + c'] \vee [l - c = l' - c'])$
- 将 $l + c, l - c$ 相同的确定必胜格按 c 排序并预先储存，容易使用 bfs 寻找任意该序列；您必获胜当且仅当存在任意该序列。

F - Grid Survival

- 现在我们需要找到一个确定必胜格序列，使所用的激活格总代价最小，只需将上文的 bfs 略作改动：
 - 记每个确定必胜格来自哪个激活格及其权值，以及到达之所需的最小总权值 dis ；
 - 按 dis 升序使用当前（或曾今）轮廓线上的确定必胜格；
 - 注意来自同激活格且同色的确定必胜格互相间可直达，共享 dis ，确定一者 dis 则另一者的 dis 要同步立刻更新并入队，不能等原值出队。

F - Grid Survival

- 时间复杂度 $O(\sum k \log k)$, 空间复杂度 $O(n + m + \sum k)$ 或 $O(\sum k)$ (会牺牲时间常数)。
- 一些小细节:
 - 超出网格范围的确定必胜格可保留, 也可删去, 因其产生的效果会被相反方向生成的确定必胜格覆盖; 但保留时注意不要越界。
 - 两颜色所用的所有确定必胜格/射线互不干扰, 可以在同一次 bfs 中求出答案。

G - Line of Sight

题意

- 给定一整点逆时针点序 n 点简单多边形 P , 和 P 内两整点 A 和 B , 判断:
 - 第一问: A, B 与 P 各个顶点的连线是否除连线端点与多边形不交;
 - 第二问: 是否存在一 P 内点 C 与 A, B 的连线除连线端点同时与多边形不交。
- $3 \leq n \leq 2 \times 10^5, |x|, |y| \leq 10^9$; 数据 $\sum n \leq 10^6$, 组数 $1 \leq t \leq 2 \times 10^5$ 。



G - Line of Sight

本题由于应研究的细节实在过多，仅粗略地讲述了思路，请见谅！

G - Line of Sight

第一问 (以 A 为例):

- 将整个图形平移, 使点 A 与原点重合而与 P 各点相对位置不变;
- 点 A 向 P 的每个顶点作一条射线, 共作出 $3 \leq m \leq n$ 条不重的射线; 将各射线极角排序, 并将平面划分为 m 个扇区; 每个扇区内部都没有 P 的顶点。
- 对于任一扇区, P 的任一边或不接触扇区内部, 或完全横跨扇区;
- 对于 P 的任一边, 可用 map 得到由边的始/终位置确定的两条射线的编号, 结合叉积还可判断边的方向和横跨哪些扇区;

G - Line of Sight

- 对于 P 的任两边，若两者共同横跨若干个扇区，则在任一“共同横跨扇区”内边之间都能以叉积比较出与原点的“远近”（由原点在扇区内方向发出的射线由哪条边遮挡决定，而非原点到边的距离，以下同），且在不同“共同横跨扇区”内“远近关系”不变。
- 若能求出原点向每个扇区的“最近边”，则对于每点，类似地取其所邻两扇区的边中，在该点对应射线上“远近关系”更近的那条边（可能无法区分，这时无论取两条边结果都一样）；该点与原点的连线除连线端点与多边形不交，**当且仅当**该点与所取的边相接。
- 由此，为求出原点向每个扇区的“最近边”，给出两种写法分支：



G - Line of Sight

- 写法一：从任意扇区其遍历每个扇区，共一圈；在易求的时机用 set 实时修改每条边是否横跨当前扇区（每条边的状态最多被修改 2 次，set 最多被修改 $O(n)$ 次）；set 内的自定义排序将自动每条边按到原点的远近关系排好，取 set 中的极值轻松得解；

G - Line of Sight

- 写法二 (std): 称从起点到终点绕原点逆时针移动的边为正向边, 逆时针则为反向边, 角度不变为无向边; 用 set 维护连续扇区的相同“最近边”; 从原点发出任意一条射线, 并找到由该射线直射的最近正向边, 以其起点为起点 S , 假想一动点 F 沿多边形正方向遍历多边形一圈; 另维护一初始为 S 对应角度的“更新角度”, 用绕数与射线编号刻画更新条件; **当且仅当** F 绕原点的绕数与射线编号与“更新角度”一致, 且在接下来遍历的扇区不被所有已遍历的边遮挡 (所遍历边在扇区中较其他所有边离原点最近) 时, F 带着“更新角度”移动并产生新的扇区“最近边”; 最后必能找到所有扇区“最近边”。
- 至此第一问的答案, 而第二问的做法一致。

G - Line of Sight

第三问:

- 称多边形上能被 A “看到”的点集为 S_A , 能被 B “看到”的点集为 S_B ;
- “存在 S_A 中的两点与 S_B 中的两点在多边形上交叉”(“四点交叉”)与“存在某 P 内点 C 同时看到 A 和 B ”之间可互相转化;
- 当 P 的某顶点由 A, B 都可看到, 显然 C 存在;
- 否则可由一次简单的线性 dp 判断顶点间是否产生“四点交叉”, “四点交叉”存在则 C 存在;

G - Line of Sight

- 否则对于 S_A ，找到其中最靠负方向的 P 顶点 P_{l_A} ，和最靠正方向的 P 顶点 P_{r_A} ；暴力查找最近的能横跨 $P_{l_A} - A - P_{r_A}$ 扇区的边（扇区内不可能有暴露的点），用叉积判：从该边起点加上该边向量的多少倍可以到达两射线与该边的交点位置（这个数值是一个分子分母均可达 10^{18} 量级的分数，值域为 $[0, 1]$ ；当值为 1 时将其视为在下一条边上）；对于 S_B 同理；

G - Line of Sight

- 此时再有某可非顶点 P' 由 A, B 都可看到时将其忽略，因无对应的 C 存在；
- 在对新点正确排序（注意使用 `int128`）后，再做一次简单的线性 dp 判断所有点间是否产生“四点交叉”，“四点交叉”存在则 C 存在，否则保证 C 不存在。
- 总复杂度 $O(n \log n)$ 。

H - VI Civilization

- 二分 p 并验证是否存在可行策略。
- 验证的时候使用动态规划。定义 $dp[i][j][z]$ 表示在前 i 回合，已完成前 j 个科技，且剩余 z 回合可用于分配生产力时，能获得的最大科技胜利点。记 $sum_i = m + \sum_{j=1}^i k_j$ ，状态转移分为三种情况：

1. 投入科技胜利点， $dp[i][j][z] + m + sum_j \rightarrow dp[i+1][j][z+1]$
2. 不触发尤里卡完成下一个科技： $dp[i][j][z] \rightarrow dp[i + \lceil \frac{a_{j+1}}{sum_j} \rceil][j+1][z + \lceil \frac{a_{j+1}}{sum_j} \rceil]$
3. 触发尤里卡完成下一个科技：

$$dp[i][j][z] \rightarrow dp[i + \lceil \frac{a_{j+1} - c_{j+1}}{sum_j} \rceil][j+1][z + \lceil \frac{a_{j+1} - c_{j+1}}{sum_j} \rceil - \lceil \frac{b_{j+1}}{p} \rceil],$$

$$z + \lceil \frac{a_{j+1} - c_{j+1}}{sum_j} \rceil - \lceil \frac{b_{j+1}}{p} \rceil \geq 0$$

H - VI Civilization

- 初始化 $dp[0][0][0] = 0$ ，其余为负无穷。通过状态转移计算所有可能状态，若存在 $i \leq t$ 使得 $dp[i][j][z] \geq s$ ，则当前 p 可行。
- 时间复杂度为 $O(t^2 n \log b)$ 。

I - Block Combination Minimal Perimeter

题意

- 你有 $i = 1 \sim n$ 的 $1 \times i$ 的矩阵各一个，你现在要将他们拼成一个实心矩形（无重叠且不能不用），求拼成矩形的最小周长。
- $1 \leq n \leq 2 \times 10^5$ 。

I - Block Combination Minimal Perimeter

- 设最终矩形边长为 a 和 b 。显然，面积一定时， $\max(a, b)$ 越小，矩形周长越小。
- 观察可得，由于大小 $1 \times n$ 积木的存在， $\max(a, b) \geq n$ 。
 1. 当 n 为奇数时，一定可以构造 $\max(a, b) = n$ 的矩形：
每行分别为 n ， $(n-1)+1$ ， $(n-2)+2$ ，...
 2. 当 n 为偶数时，构造不出 $\max(a, b) = n$ 矩形：
 $1 \times n$ 的积木肯定单独一行， $1 \times (n-1)$ 的积木只能和 1×1 的积木拼一行才不会超过 n 。同理， $1 \times (n-2)$ 只能和 1×2 ， $1 \times (n-3)$ 只能和 1×3 ，...最后剩下一个 $1 \times \frac{n}{2}$ 的积木无法配对。

I - Block Combination Minimal Perimeter

3. 当 n 为偶数时，一定可以构造 $\max(a, b) = n + 1$ 的矩形：

每行分别为 $n + 1$, $(n - 1) + 2$, $(n - 2) + 3$, ...

■ 综上所述，最小矩形周长为 $2(n + 1 + \lfloor \frac{n}{2} \rfloor)$ 。

J - Fastest Coverage Problem

题意

- 给定一 $n \times m$ 的 01 网格，你可再任选至多一个 0 变为 1；之后每秒每个与 1 相邻的 0 都会变为 1，最小化整个网格变为 1 的用时。
- $1 \leq n \times m \leq 2 \times 10^5$ 。

J - Fastest Coverage Problem

- 我们可以自然地由二分答案钦定答案为 k ，判其是否有解；
- 可以先 *BFS* 求出所有点到 1 的最短距离，然后可以快速得到所有还未被覆盖的点。二分的 *check* 可以考虑枚举所有点作为改变的点，然后等价于求其到所有未被覆盖点最大距离。
- 可以发现，最大距离只需关注这些网格的 $\max(y+x), \min(y+x), \max(y-x), \min(y-x)$ 4 个值；
- 复杂度 $O(nm \log(n+m))$ ，最优可以做到 $O(nm)$ ，但容易出错需要讨论奇偶性，场上错误的代码大概都可以被 2×2 的全 0 网格 *hack*。

K - Perfect Journey

题意

- 给出一颗 n 个节点树，以及 m 条关键边， k 条树上路径，你需要从中选出最少的路径使得所有关键边被经过，输出最少选几条和方案数。
- $3 \leq n \leq 2 \times 10^5$, $1 \leq m \leq 22$, $1 \leq k \leq 2 \times 10^5$ 。

K - Museum Acceptance

题解

- 可以用状压来表示经过边的情况，这样等价于选择最少的路径使得或起来是全集。
- 可以考虑使用 *fwT* 来解决这个问题，直接做是 $O(m^2 2^m)$ ，但是我们每次只关注全集项的系数，只需要通过容斥来计算该项系数。
- 记 $mask = 2^m - 1$, f 为做 t 次 *fwT* 后的多项式，计算全集项为
$$\sum_{i=0}^{2^m-1} (-1)^{\text{pop_cnt}(i \oplus mask) \& 1} \times f_i$$
- 复杂度 $O(m 2^m)$ ，此外有依赖于树的结构 $O(m^2 2^m)$ 和 $O(m 2^m)$ 做法，前者可能无法通过本题。

L - Float

题意

- 现有两个数值 $a = 0$ 和 $b = z$ ，目标为使 $a = n$ 。你每次尝试有 p 的概率成功使 a 加一，另有 $1 - p$ 的概率失败：若 b 不为零使其减一，否则使 a 归零。对于每个 $z \in [0, m]$ ，问期望尝试次数。
- $1 \leq n \leq 10^9, 1 \leq m \leq 2 \times 10^5, 0 < p < 998244353$ ；答案对 998244353 取模。

L - Float

- 记 b 已减一 i 次, $a = j$ 的状态为 (i, j) ;
 - 以下讨论 $i \geq 0, 0 \leq j < n$ 的状态;
 - 当 $i < z$ 时, (i, j) 有 p 的概率转移至 $(i, j+1)$, 另有 $1-p$ 的概率转移至 $(i-1, j)$;
 - 当 $i = z$ 时, (i, j) 有 p 的概率转移至 $(i, j+1)$, 另有 $1-p$ 的概率转移至 $(i, 0)$ 。
- 将状态转移的路径分为未出现过 a 归零的“第一部分”和已出现过 a 归零的“第二部分”, 分别计算两部分的期望步数, 最后求和。

L - Float

■ 第一部分:

- 每个 $0 \leq i \leq z, 0 \leq j < n$ 的状态在第一部分中最多被经过（作为起点）一次，第一部分的期望步数为这些状态被经过（作为起点）的概率之和；
- 设状态 (i, j) 被经过（作为起点）的概率为 $P_{i,j}$ ，考虑由成功/失败而来的两种转移来源，有：

$$P_{i,j} = \begin{cases} 1, i = j = 0 \\ p^j, [i = 0] \wedge [0 \leq j < n] \\ P_{i,j-1} \cdot p + P_{i-1,j} \cdot (1 - p), [1 \leq i \leq z] \wedge [0 \leq j < n] \\ 0, otherwise \end{cases}$$

L - Float

- 进一步地, 设 $S_i = \sum_{j=0}^{n-1} P_{i,j}$;
- 对于 $i = 0$, 由等比系数求和公式有 $S_i = \sum_{j=0}^{n-1} p^j = \frac{1-p^n}{1-p}$;
- 对于 $i > 0$, 由 $P_{i,j} = P_{i,j-1} \cdot p + P_{i-1,j} \cdot (1-p)$
得 $\sum_{j=0}^{n-1} P_{i,j} = \sum_{j=0}^{n-1} P_{i,j-1} \cdot p + \sum_{j=0}^{n-1} P_{i-1,j} \cdot (1-p)$,
即 $S_i = (S_i - P_{i,n-1} + P_{i,-1}) \cdot p + S_{i-1} \cdot (1-p)$,
整理得递推式 $S_i = S_{i-1} - P_{i,n-1} \cdot \frac{p}{1-p}$;

L - Float

- 而 $P_{i,n-1} = \frac{(n-1+i)!p^{n-1}(1-p)^i}{i! \cdot (n-1)!}$, 有 $P_{0,n-1} = p^{n-1}$
和递推式 $\forall i > 0 \rightarrow P_{i,n-1} = P_{i-1,n-1} \cdot \frac{(n-1+i)(1-p)}{i}$ 。
- 于是 $P_{i,n-1}$ 和 S_i 都可递推, 求每个 z 的第一部分答案 $\sum_{i=0}^z S_i$ 复杂度为
 $O(m \log m)$ (来自逆元) 或 $O(m)$ (优化为线性逆元)。

L - Float

■ 第二部分:

- 进入第二部分的概率，考虑枚举 $b = 0$ 且失败时 a 的值，为

$$\sum_{j=0}^{n-1} P_{z,j} \cdot (1-p), \text{ 即 } S_z \cdot (1-p);$$

- 至于进入第二部分后的期望步数，对于各 z 相同，记其为 C ，现尝试求 C :

- 设从 $a = 0$ 到 $a = j$ 的期望步数为 f_j , $f_0 = 0$;
- 对于 $j > 0$, 考虑从 $a = j-1$ 用 1 步尝试到 $a = j$, 有 p 的概率成功, 有 $1-p$ 的概率从 $a = 0$ 重新开始到 i ,

得式子 $f_j - f_{j-1} = 0 \cdot p + (f_j - f_0) \cdot (1-p) + 1$,

整理得 $f_j = \frac{f_{j-1}+1}{p}, f_j - \frac{1}{p-1} = \frac{1}{p}(f_{j-1} - \frac{1}{p-1}) = (\frac{1}{p})^j(-\frac{1}{p-1}), f_j = \frac{1-p^j}{(1-p)p^j}$;

- 则 $C = f_n = \frac{1-p^n}{(1-p)p^n}$ 。

L - Float

- 最后，对于每个 $z \in [0, m]$ ，输出 $\sum_{i=0}^z S_i + S_z \cdot \frac{1-p^n}{p^n}$ 。
- 总复杂度 $O(m \log m)$ 或 $O(m)$ 。

M - Mysterious Spacetime

题意

- 你有 n 个时间 t_i 会出现 $[l_i, r_i]$ 的所有能量, 保证所有时间不同。
- m 个生成物会在满足区间交集大于 k_i 的时刻生成。
- q 个询问 $tl_i\ tr_i$ 时间内最早的时间有被区间 $[L_i, R_i]$ 包含的生成物生成。
- $1 \leq n \leq 5 \times 10^5, 1 \leq m \leq 5 \times 10^5, 1 \leq q \leq 5 \times 10^5$;

M - Mysterious Spacetime

- 对于区间交问题的可以把区间的形态分为四种，考虑所有能对 $[l, r]$ 满足交集大于 k 线段 $[L, R]$ 长什么样子：
 1. $l \leq L \leq r \leq R$
 2. $L \leq l \leq R \leq r$
 3. $l \leq L \leq R \leq r$
 4. $L \leq l \leq r \leq R$
- 第 4 种显然满足，可以按右端点倒着扫描然后在线段树上加入左端点，询问时可以直接查询 $[1, r - k + 1]$ ，这样子就可以把 2, 4 解决了。
- 剩下 1, 3 可以考虑按照长度降序扫描，在线段树 R 上加入，对于询问查询区间 $[l - k + 1, r]$ 即可。

M - Mysterious Spacetime

- 求出了每个生成物的 t_i 之后，我们把询问 L, R 当作二维平面上的点，按照 t 升序，每次一个生成物 t_i, l_i, r_i 等价于把所有询问包含这个这个区间的全部赋值成 t_i 然后删掉。等价于单点加，清空一个矩形。
- 可以对每个左端点开个 *set* 记录右端点集合，线段树上存储 l 的 $maxr$ ，那么可以借助线段树二分，在 $[1, l]$ 找到满足大于等于 r 的下标，然后赋值，删除对应询问更新线段树，每个节点只会进入和删除一次。
- 总的复杂度 $O((n + m + q) \log x)$ ，常数比较小的 $2 \log$ 也可以通过。

THANKS!

AC.NOWCODER.COM